

□ Unidad 1 – Modelo de Response, Código de Estado y Manejo de Errores en FastAPI

1□ Fundamentos: ¿Qué es un Modelo de Respuesta?

◆ ¿Qué es un modelo de respuesta?

El modelo de respuesta (response model) define la estructura y el formato de los datos que el servidor envía de vuelta al cliente.

Cuando el cliente hace un request, el servidor procesa la solicitud y devuelve una respuesta HTTP con un cuerpo en formato JSON. El `response_model` es el contrato que describe cómo debe lucir ese JSON.

◆ ¿Para qué sirve el `response_model`?

- □ Filtrado de datos (seguridad): elimina campos que no deben exponerse al cliente.
- □ Conversión de datos (serialización): transforma objetos de Python/ORM a JSON válido.
- □ Documentación automática: genera el JSON Schema para Swagger y Redoc.

□ *FastAPI usa el tipo de retorno (o el parámetro `response_model`) para validar, documentar y serializar la respuesta, y sobre todo para limitar y filtrar los datos de salida.*

2□ El parámetro `response_model`

Puedes declarar el tipo usado para la respuesta anotando el tipo de retorno de la path operation function, o usando el parámetro `response_model` del decorador.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None
    tags: list[str] = []

@app.post("/items/")
async def create_item(item: Item) -> Item:
    return item
```

Existen casos en los que necesitas devolver datos que no son exactamente lo que declara el tipo (por ejemplo, un dict o un objeto de base de datos) pero quieres documentarlo como un modelo de Pydantic. Para eso se usa el parámetro `response_model`:

```
from typing import Any
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
```

```
@app.post("/items/", response_model=Item)
async def create_item(item: Item) -> Any:
    return item
```

❑ Modelo de entrada vs. modelo de salida

Un caso de uso clave: separar el modelo que recibe datos sensibles (por ejemplo una contraseña) del modelo que se devuelve al cliente.

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn):
    return user # FastAPI filtra la contraseña automáticamente
```

❑ *Nunca devuelvas una contraseña en texto plano en un response. UserOut existe justamente para que FastAPI filtre ese campo antes de enviarlo al cliente.*

3❑ Comparativa: Tipo de Retorno vs. response_model

Característica	-> list[Item] (solo tipo)	response_model=... (parámetro)
❑ Rol principal	Tipado para Python / IDE	Contrato de salida de FastAPI
❑ Responsabilidad	Devuelves instancias del modelo	FastAPI adapta tu salida al modelo
❑ Datos admitidos	Deben ser instancias del modelo	Puede ser dict, list u objeto ORM
❑ Filtrado automático	✗ No (lo hacés vos)	✓ Sí (elimina campos extra)
❑ Validación de salida	✗ No automática	✓ Sí (estructura y tipos)
❑ OpenAPI / Swagger	❑ No siempre alcanza	✓ Genera schema y documentación
❑ Soporte del editor	✓ Máximo (tipo real conocido)	❑ Limitado si retornás Any

❑ Lo mejor de los dos mundos: herencia de clases

Usando clases base y herencia se puede anotar el tipo de retorno para el editor y mypy, y a la vez obtener el filtrado de datos de FastAPI:

```
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr
```

```
app = FastAPI()

class BaseUser(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

class UserIn(BaseUser):
    password: str

@app.post("/user/")
async def create_user(user: UserIn) -> BaseUser:
    return user
```

□ *BaseUser es la clase base y UserIn hereda de ella agregando password. FastAPI filtra la salida usando solo los campos de BaseUser, aunque la función devuelva un UserIn completo.*

4□ Código de Estado del Response

De la misma manera que se declara un `response_model`, se puede declarar el código de estado HTTP usado en la respuesta con el parámetro `status_code` del decorador.

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

◆ Acerca de los códigos de estado HTTP

- 100–199 → Información. Rara vez se usan directamente.
- 200–299 → Éxito. 200 es el valor por defecto ("OK"); 201 es "Created"; 204 es "No Content" (sin body).
- 300–399 → Redirección. 304 "Not Modified" no debe tener body.
- 400–499 → Error del cliente. 404 "Not Found" es el más conocido; 400 sirve para errores genéricos.
- 500–599 → Error del servidor. Se generan automáticamente cuando algo falla en el código de la aplicación.

5□ Atajo con `fastapi.status`

No hace falta memorizar el número de cada código: FastAPI expone variables de conveniencia en `fastapi.status` para usar el autocompletado del editor.

```
from fastapi import FastAPI, status

app = FastAPI()

@app.post("/items/", status_code=status.HTTP_201_CREATED)
async def create_item(name: str):
    return {"name": name}
```

□ *status_code también puede recibir un `IntEnum`, como `http.HTTPStatus` de la librería estándar de Python. `status.HTTP_201_CREATED` proviene en realidad de `starlette.status`; FastAPI solo lo re-exporta por comodidad.*

6□ Excepciones en Python

Una excepción es un mecanismo de Python para señalar que ocurrió una situación anómala (un error o un caso no válido) y cortar el flujo normal de ejecución.

- ✓ Si todo va bien: la función sigue su curso y eventualmente hace return.
- ✗ Si ocurre un problema: se lanza una excepción con raise.
- □ Al lanzarse, se detiene el flujo normal y Python busca quién la maneje (try/except).
- □ Si nadie la maneja, el programa (o la petición) falla.

7 □ Manejo de errores con HTTPException

Para notificar errores al cliente (falta de privilegios, recurso inexistente, etc.) normalmente se devuelve un código de estado HTTP en el rango 400–499. FastAPI usa HTTPException para esto.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
items = {"foo": "The Foo Wrestlers"}

@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item": items[item_id]}
```

◆ Puntos clave

- □ Códigos 4xx: se usan cuando el problema está en la solicitud del cliente (400–499).
- □ HTTPException es una excepción de Python: se lanza con raise, nunca se devuelve con return.
- □ Detiene la ejecución del endpoint y responde inmediatamente.
- □ Incluye status_code (ej. 404) y detail (mensaje para el cliente); detail puede ser dict o list, no solo str.

□ Respuesta estándar en JSON

- ✓ Si existe → 200 con { "item": ... }
- ✗ Si no existe → 404 con { "detail": "Item not found" }

8 □ Agregar headers personalizados

En escenarios avanzados (por ejemplo, ciertos tipos de seguridad) es útil poder agregar headers personalizados al error HTTP.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()
items = {"foo": "The Foo Wrestlers"}

@app.get("/items-header/{item_id}")
async def read_item_header(item_id: str):
    if item_id not in items:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
            headers={"X-Error": "There goes my error"},
        )
    return {"item": items[item_id]}
```

9 Manejadores de excepciones personalizados

Se pueden instalar manejadores de excepciones propios con `@app.exception_handler()`, reutilizando las utilidades de excepciones de Starlette. Esto permite manejar globalmente excepciones definidas por el desarrollador.

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

class UnicornException(Exception):
    def __init__(self, name: str):
        self.name = name

app = FastAPI()

@app.exception_handler(UnicornException)
async def unicorn_exception_handler(request: Request, exc: UnicornException):
    return JSONResponse(
        status_code=418,
        content={"message": f"Oops! {exc.name} did something."},
    )

@app.get("/unicorns/{name}")
async def read_unicorn(name: str):
    if name == "yolo":
        raise UnicornException(name=name)
    return {"unicorn_name": name}
```

❑ *FastAPI tiene su propio `HTTPException`, que hereda del `HTTPException` de Starlette y acepta cualquier dato JSON-able en detail (no solo str). Al registrar un manejador propio para `HTTPException`, conviene registrarlo sobre `StarletteHTTPException`, para capturar también las excepciones que puedan lanzarse desde el propio Starlette.*

❑ Errores Comunes (muy importante para estudiantes)

- 1. Usar `return` en vez de `raise` al lanzar un `HTTPException`.
- 2. Olvidar el `status_code` y depender siempre del 200 por defecto.
- 3. No usar las constantes de `fastapi.status` y arriesgarse a errores de tipeo.
- 4. Devolver información sensible (contraseñas, tokens) por no declarar un modelo de salida distinto al de entrada.
- 5. Registrar un manejador para `HTTPException` de FastAPI en vez de `StarletteHTTPException`.
- 6. No declarar tipos de retorno, perdiendo validación, documentación y filtrado automático.

❑ Resumen General

- ✓ El `response_model` define el contrato de salida de la API: valida, documenta y filtra.
- ✓ El tipo de retorno de la función y el parámetro `response_model` cumplen roles complementarios.
- ✓ `status_code` (o `fastapi.status`) declara el código HTTP devuelto y se documenta en OpenAPI.
- ✓ 2xx significa éxito; 4xx significa error del cliente; 5xx, error del servidor.
- ✓ `HTTPException` se lanza con `raise`, corta el flujo y responde con `{ "detail": ... }`.
- ✓ Es posible personalizar headers de error y crear manejadores de excepciones propios.

❑ Recursos para Estudiar

- ► Video de la clase: <https://www.youtube.com/watch?v=8ncgLESDsbU>
- ❑ Modelo de Response: <https://fastapi.tiangolo.com/es/tutorial/response-model/>

-
- ☐ Código de Estado: <https://fastapi.tiangolo.com/es/tutorial/response-status-code/>
 - ☐ Manejo de Errores: <https://fastapi.tiangolo.com/es/tutorial/handling-errors/>